

UC LINA 600 LINEAR ALGEBRA FOR GRAPHICS · MODULE 13 · LECTURE NOTES

Claiming a Transform Is Correct

Verification, rather than looking at the picture.

MODULE OUTCOMES

- Explain why visual inspection is insufficient.
- Build a verification suite for transforms.
- State the invariants worth checking.
- Assess whether you are ready for Semester 1.
- State the program's closing rule in this subject.

1 Why looking is not enough

The error	What you actually see
Transposed rotation matrix	Everything rotates the wrong way — which looks deliberate, and reads as a control-direction preference.
Wrong handedness convention (Module 06 §3)	A mirrored scene, which reads as an art choice until somebody tries to read text in it.
Normals transformed by M rather than the inverse transpose (Module 06 §4)	Slightly wrong shading, but only under non-uniform scale — so it passes every uniformly-scaled test.
Normals not renormalised after transforming	Too bright or too dark — which reads as a lighting parameter problem and gets 'fixed' by changing the light.
Composition order swapped (Module 03 §2)	Correct results for symmetric test scenes only — see the note.

A symmetric test scene hides composition errors — a cube at the origin rotated about its own centre looks identical under several wrong orderings — **which is why the test scene should be deliberately asymmetric and at a known off-axis position.** Every row in this table is a bug that ships, because each one produces a picture, and a picture is what most people check against.

BECAUSE A PLAUSIBLE IMAGE IS THE WEAKEST POSSIBLE EVIDENCE

The pipeline will produce a picture from almost any set of matrices — so 'I see something' rules out only crashes and all-black output, and crashes were never the risk.

And you are the worst possible judge of your own scene — you know what it is supposed to look like, so you see what you expect to see (CSCE 671 Module 01 §2's designer blindness, and UC MATH 600 Module 13 §2's inability to find gaps in your own argument — the same phenomenon twice).

So the fix is numbers: a point whose screen position you computed by hand on paper, asserted automatically — which cannot be talked out of a disagreement and does not know what the scene is supposed to look like.

Which takes about an hour to set up and saves days — and is the one engineering habit this course is trying to install, over and above the mathematics.

2 The verification suite

```
1  KNOWN POINTS
    a unit cube at the origin, a camera on an
    axis, and screen positions you computed on
    paper. Assert them exactly.

2  ROUND TRIPS
    M * M^-1 == I to a stated tolerance.
    world_from_camera * camera_from_world == I.

3  INVARIANTS OF ROTATIONS
    orthogonal:  R * R^T == I
    determinant: det(R) == +1
                  (Module 09 section 3)
    lengths:    |Rv| == |v|
    -- three cheap checks that between them catch
        transposes and handedness errors

4  DEGENERACY GUARDS
    |det| above tolerance before inverting
    |n| above tolerance before normalising
    (Module 06 section 2)

5  AND A VISUAL CHECK LAST
    normals drawn as segments, basis axes drawn
    in colour -- useful for orientation, and not
    evidence of correctness.

THE FIRST THREE WOULD CATCH EVERY ROW OF THE TABLE
IN SECTION 1.
```

The first three checks would catch every failure in §1's table — which is the argument for writing them, and they are about forty lines of code total. Project 2 requires this suite and at least one bug

found by it rather than by looking at the image, because finding one is the demonstration that the suite is doing work.

3 Are you ready?

- **Can you derive a rotation matrix by drawing where the basis vectors go**, rather than recalling the sign pattern? (Module 03 §1.)
- **Can you derive the projection-onto-a-vector formula from the perpendicularity condition**, in three lines? (Module 05 §2.)
- **Can you say why the view matrix is an inverse, and why for an orthonormal frame that inverse is a transpose?** (Module 04 §3.)
- **Can you explain what the fourth coordinate is for — both of its jobs**, making translation linear and enabling the perspective divide? (Module 07.)
- **And can you derive the perspective projection matrix from similar triangles?** (Module 08 §2.) **Five questions. Answer all five and start Semester 1; otherwise the ones you missed name exactly the modules to revisit**, which is more useful than a mark.

4 The rule

STATE WHAT YOU VERIFIED, STATE WHAT YOU ASSUMED, AND NEVER CLAIM MORE THAN YOU ESTABLISHED

In this subject it means three things: name the handedness convention, name the spaces each matrix maps between (Module 04 §2's from-naming), and say what you actually tested.

'The transform is correct' means 'it passes these assertions under this convention' — which is a checkable claim — unlike 'it looks right', which is a report about a person rather than about a matrix.

And the convention is the thing most often left unstated — which is why essentially every handedness bug is one person's unstated assumption meeting another person's unstated assumption, usually across a file format boundary.

So: convention, spaces, tests — and that is the prerequisite done, closing on the same rule the other thirty-six courses close on, which it turns out applies to a 4×4 matrix as readily as to an experiment.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Gortler, the conventions appendix (library copy)	§4 — conventions stated explicitly, which is what this module is asking you to do.

Resource	What it covers for this module
Scratchapixel, the testing and debugging notes (free)	§2 — practical verification of a renderer, free.
Shirley — Ray Tracing in One Weekend (free)	§§1 and 2 — a complete small renderer to test your understanding against, free and short.
The glTF specification, the coordinate conventions (free)	§4 — a format that states its conventions explicitly, which is why it interoperates.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Introduce each of §1's five errors** deliberately and look at the result.
- 2 **Build an asymmetric test scene** and show it catches a composition error the symmetric one missed.
- 3 **Compute one point's screen position by hand**, through all four stages.
- 4 **Assert it** in a test.
- 5 **Write the round-trip tests** for every matrix you build.
- 6 **Write the three rotation invariant checks.**
- 7 **Verify they catch a transposed rotation.**
- 8 **Add the degeneracy guards** and feed them a degenerate triangle.
- 9 **Take the five readiness questions** without reference.
- 10 **Project 2 is now due.** Submit the renderer, the derivation of every matrix in it, the verification suite, at least one bug the suite found rather than your eyes, and a statement of the handedness convention you used.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name five wrong transforms that still produce a picture.
- 2 Why does a symmetric test scene hide errors?
- 3 Why are you the worst judge of your own scene?
- 4 What is the fix, and why does it work?
- 5 Give the five categories of check, in order.
- 6 Which three would catch everything in the table?
- 7 Give the three rotation invariants.

- 8 What are the degeneracy guards for?
- 9 Give the five readiness questions.
- 10 State the closing rule in this subject's terms.